

ZPR Projekt - Dokumentacja Końcowa

Narzędzie CLI do wektoryzacji grafiki rastrowej

Karol Adamski (331155)
Bartłomiej Masiak (331176)

Politechnika Warszawska, WEiTl

6 czerwca 2026

Spis treści

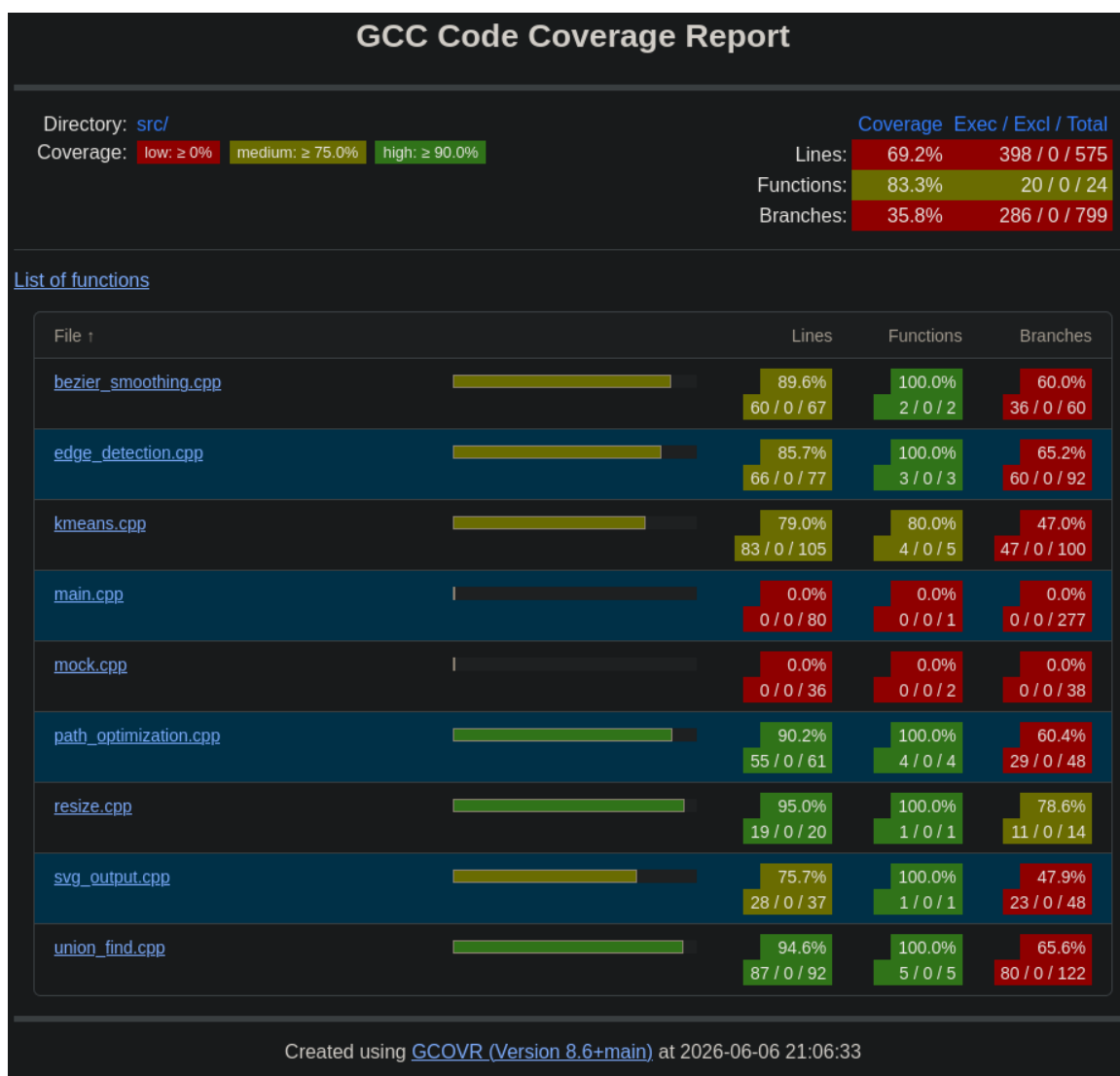
1. Metryki Projektu	2
2. Architektura i Jakość Kodu	2
3. Realizacja Założeń Wstępnych	3
4. Testy Jednostkowe i Wieloplatformowość	3
5. Retrospekcja i Napotkane Problemy	3
6. Weryfikacja Wizualna Wyników	4

1. Metryki Projektu

Poświęciliśmy łącznie 42 godziny na realizację projektu. Prace nad architekturą, implementacją oraz testami zajęły 36 godzin. Przygotowanie dokumentacji wstępnej i końcowej zajęło 6 godzin.

Za pomocą polecenia `cloc` zliczyliśmy rozmiar bazy kodu. Projekt składa się z 1729 linii kodu źródłowego. Przygotowaliśmy 20 unikalnych scenariuszy testowych, zgrupowanych w 7 głównych zestawach.

Zmierzyliśmy pokrycie kodu testami za pomocą narzędzia `gcovr`. Raport z działania programu wskazuje na 69.2 procent pokrycia wykonanych linii kodu.



Rys. 1: Raport pokrycia kodu wygenerowany przez narzędzie `gcovr`.

2. Architektura i Jakość Kodu

Podzieliśmy potok wektoryzacji na logiczne przestrzenie nazw, między innymi `Stage::KMeans`, `Stage::EdgeDetection` oraz `Stage::SvgOutput`.

Wykorzystaliśmy mechanizmy standardu C++23 oraz bibliotekę STL do zarządzania pamięcią. Wymusiliśmy jednolity styl formatowania plików źródłowych za pomocą profilu narzędzia `clang-format`. Wygenerowaliśmy dokumentację interfejsu programistycznego używając systemu `Doxygen`.

3. Realizacja Założeń Wstępnych

Zrealizowaliśmy wszystkie założenia zdefiniowane w dokumentacji wstępnej. Zaimplementowaliśmy i przetestowaliśmy pełną ścieżkę konwersji obrazów rastrowych do formatu wektorowego SVG.

Kompletna instrukcja obsługi oraz instrukcja budowania projektu znajdują się w pliku `README.md` w głównym katalogu repozytorium.

4. Testy Jednostkowe i Wieloplatformowość

Zautomatyzowaliśmy proces testowania za pomocą frameworka `Catch2`. Wykorzystaliśmy mechanizm tworzenia fałszywych struktur danych obrazu do izolowanego testowania logiki segmentacji.

```
1 (venv) ~/L/img-svg $ ./build/run-tests --list-tests
2 All available test cases:
3   EdgeDetection stage traces contours of connected regions
4     [edge_detection]
5   SvgOutput stage writes vector data to a valid SVG file
6     [svg_output]
7   UnionFind stage groups pixels and eliminates noise
8     [union_find]
9   BezierSmoothing stage calculates curves based on corner angles
10    [bezier_smoothing]
11   KMeans stage partitions image pixels into K clusters
12    [kmeans]
13   PathOptimization stage simplifies polygons using RDP algorithm
14    [path_optimization]
15   Resize stage downscales images based on max_width
16    [resize]
17 7 test cases
```

Listing 1: Lista zaimplementowanych przypadków testowych w narzędziu `Catch2`.

Zweryfikowaliśmy proces budowania i działania potoku przetwarzania w dwóch środowiskach systemowych. Projekt kompiluje się bez błędów pod systemem operacyjnym Linux z użyciem kompilatora `g++` oraz pod systemem operacyjnym Windows z użyciem kompilatora `Visual C++`.

5. Retrospekcja i Napotkane Problemy

Zidentyfikowaliśmy problem z drastycznym spadkiem wydajności algorytmów dla wejściowych plików rastrowych o dużej rozdzielczości. Rozwiązaliśmy ten problem poprzez wprowadzenie modułu wstępnego skalowania obrazu na podstawie parametru `max-width`. Algorytm ten redukuje wymiary usuwając drobne szczegóły na wczesnym etapie potoku, co znacząco przyspieszyło proces wektoryzacji dla dużych obrazów.

Zaobserwowaliśmy powstawanie niechcianych szczelin pomiędzy przylegającymi wielokątami na wygenerowanych obrazach SVG. Wynikało to z faktu niezależnego aproksymowania krawędzi każdego obszaru o jednolitym kolorze. Ustaliliśmy, że eliminacja tego błędu wymagałaby całkowitej przebudowy architektury kodu. Konieczne byłoby zastosowanie globalnego grafu współdzielonych krawędzi topologicznych. Zrezygnowaliśmy z implementacji tego rozwiązania ze względu na ograniczone ramy czasowe projektu.

Istotnym ograniczeniem programu jest również jego duża wrażliwość na dobór parametrów wejściowych. Uzyskanie optymalnego rezultatu, stanowiącego kompromis pomiędzy wiernością odwzorowania a rozmiarem wynikowego pliku SVG, wymaga od użytkownika ręcznego eksperymentowania.

6. Weryfikacja Wizualna Wyników

Weryfikację wizualną potoku przeprowadziliśmy przy użyciu obrazu testowego przedstawionego na Rysunku 2. Plik wynikowy zaprezentowany na Rysunku 3 wygenerowaliśmy dla parametru maksymalnej szerokości równego 512 pikseli, tolerancji o wartości 1.2, liczby kolorów ograniczonej do 9, szerokości obrysu wynoszącej 2.5 oraz minimalnego obszaru plamy równego 4 piksele.



Rys. 2: Oryginalny obraz wejściowy poddany wektoryzacji.



Rys. 3: Obraz wynikowy wygenerowany przez stworzone narzędzie.